

ESP32-P4 IoT Fleet

End-to-End Workflow: Manufacturing → AWS IoT Core → ERP

1000-device Fleet Provisioning with claim certificates



PHASE 1 — One-Time AWS Infrastructure Setup (before manufacturing)



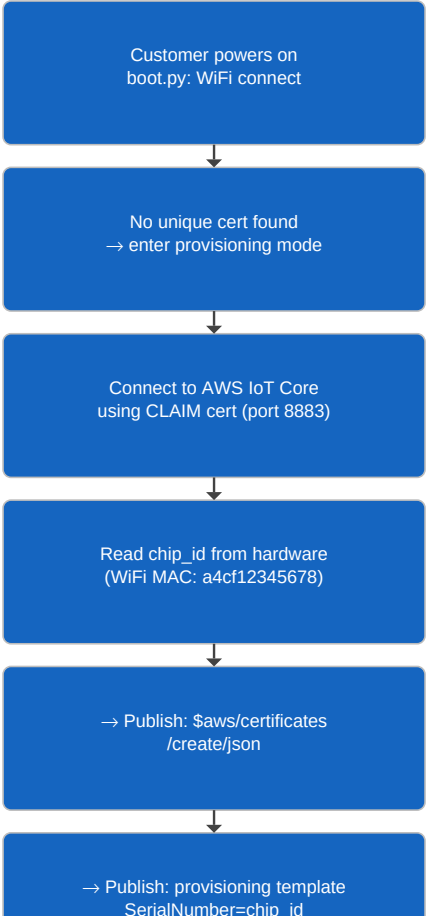
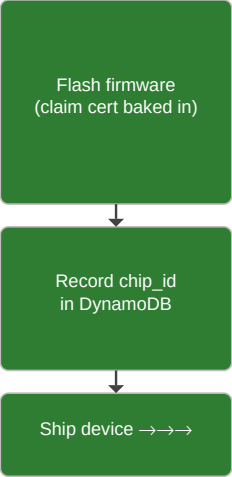
DynamoDB record — one row per device:

chip_id (PK)	48-bit hardware ID = WiFi MAC. Burned into silicon by Espressif. Cannot be changed or forged.
provisioned	false at factory → true after first boot (Lambda sets this during RegisterThing)
provisioned_at	Timestamp written by Lambda the moment the device first provisions
thing_name	AWS Thing name assigned at provision time, e.g. esp32p4-a4cf12345678
erp_id	ERP asset ID written back by the ERP Lambda after Phase 4 registration

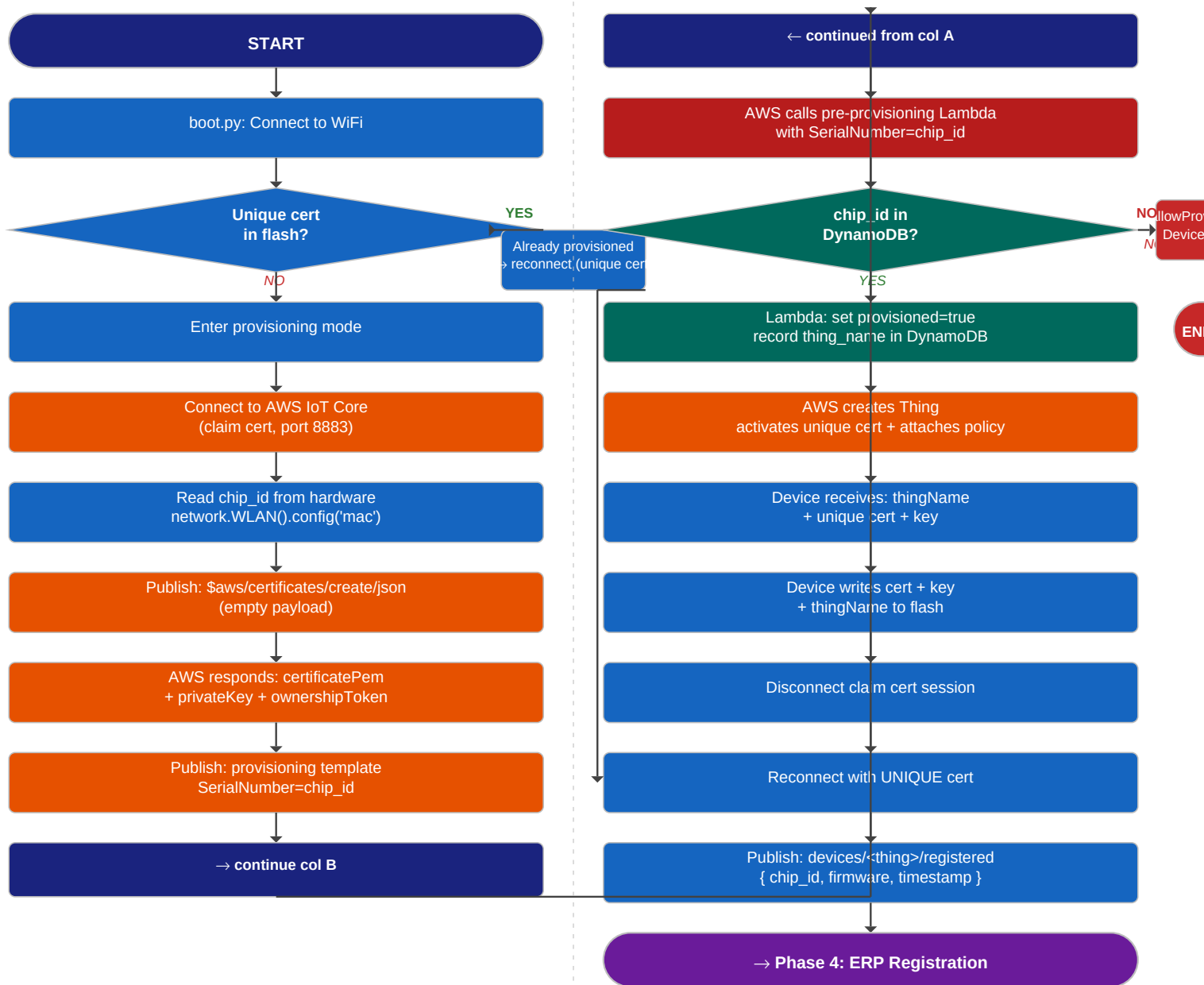
PHASE 2 — Manufacturing (repeated per batch at the factory)



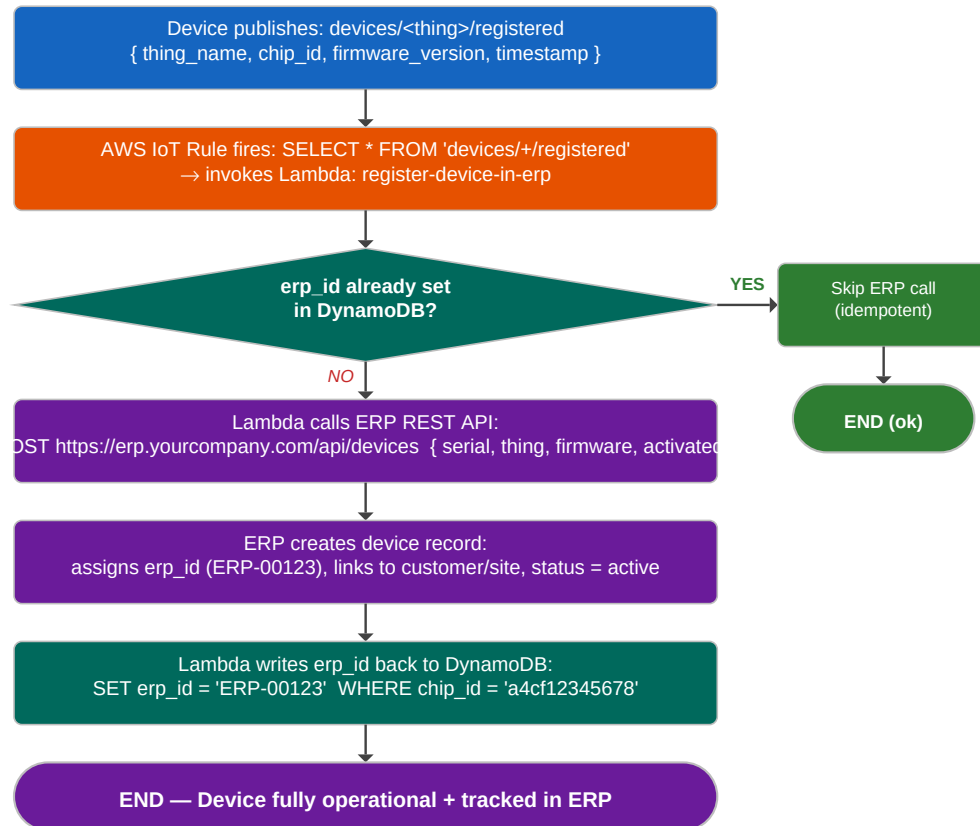
Full Workflow — Swim Lane Diagram



Phase 3 — Device First Boot: Provisioning Flowchart



Phase 4 — ERP Registration Flowchart + Idempotency



Idempotency — Handling Reboots & Retries

Reboot before cert stored

Device restarts provisioning from scratch. Lambda should deactivate incomplete certs (those with no attached Thing) to avoid orphaned certs accumulating in AWS IoT.

Reboot after cert before /registered

Device reconnects with its unique cert and republishes `/registered`. The ERP Lambda checks `erp_id` in DynamoDB before calling the ERP API — skips the call if already set.

Second provision same chip_id

Lambda finds `provisioned=true` in DynamoDB → returns `allowProvisioning=false`. AWS rejects the registration. Only one unique cert is ever issued per device.